

MICROSOFT FOUNDRY · 45 MIN

Right Model, Right Job.

End-to-End AI Development in Microsoft Foundry

Naomi — Developer · **Yina** — Decision Maker

Contoso Travel Concierge

PART 1

The problem.

Why most AI demos can't survive their first CFO email.

"Loved the demo. Two questions before we ship:

1) How do you know it's right?

2) At 8,000 trips/month, why is the bill \$47K next quarter?"

— Email from the CFO, last Tuesday

The challenge going in

*"One big frontier model.
Opaque quality."*

That's where most of us start.

It works. It's expensive. We can't defend it.

Five challenges every AI team is hitting

1 • **Model choice is exploding** — more models, providers, modalities, and deployment options than ever.

2 • **Benchmarks aren't enough** — public scores don't reflect **your** prompts, users, data, or business goals.

3 • **Cost is a system-level problem** — routing, caching, batching, prompts, tiers, observability — not just model price.

4 • **Production demands control** — reliability, monitoring, safety, governance, versioning, rollback.

5 • **The landscape keeps changing** — new models arrive constantly, can you improve app without rebuilding it?

The takeaway, coming out

*"The unit of progress is the model decision —
made per task, against a per-task scorecard."*

If you remember one sentence from this session, this is it.

We'll come back to it on the closing slide.

PART 2

Meet the workload.

One sentence from Carmen. Five jobs
hiding inside it.

Carmen

Senior engineer · Contoso San Diego office

Needs to be in **Berlin Tuesday morning** for an offsite that runs through Thursday. She messages the agent at 11 PM:

"Book flights + hotel near Alexanderplatz, under \$2,500. Also — here's a parking receipt from SAN three days ago, can you expense it?"



Five jobs hiding in one sentence

#	Job	What the model actually needs
1	Route the intent	Fast classifier · cheap · no creativity
2	Read the receipt image	Vision · structured extraction
3	Answer policy: "expense parking?"	Knowledge of Contoso policy specifically
4	Plan the trip + call tools	Strong reasoning · longer context · tool use
5	Translate the German confirm email	Multilingual · short form

*There is **no single model** that's best for all five.*

PART 3

The scorecard.

Three bars. Every step has to move at least one of them.

Our narrative device

Quality		0%	← target ≥ 0.92 on curated eval
Cost		11¢	← target $\leq 3¢$ / task
Latency		12.3s	← target $\leq 8s$ p50

These are the only numbers that matter.

*Quality, cost, latency — **per task**.*

PART 4

One model, all jobs.

The version of this we all ship first.

Starting architecture — everything on `gpt-5.4`

```
# This is the entire architecture
client.chat.completions.create(
    model="planner-gpt54",          # gpt-5.4
    messages=[system, user_message],
    tools=[search_flights, search_hotels, check_policy, submit_booking],
)
```

-  Ships in a day
-  Demos beautifully
-  **No eval** — quality is **belief**, not measurement
-  **No model choice** — one tool for every job

THE CFO MATH

\$10,560

per year, just on travel planning

\$/TASK

\$0.108

TRIPS/MONTH

8,000

QUALITY

?

Where we start — all red

Quality		?	← we don't even know
Cost		\$0.108	
Latency		12.3s	

*The bars **should** be red here. That's the point.*

This is the gap the next 30 minutes close.

PART 5

Microsoft Foundry.

The surface where models, evaluations,
and agents share a project.

What is Microsoft Foundry, concretely?

PROJECT

Workspace that ties everything together

MODELS + ENDPOINTS

What's deployed, where, at what SKU

PLAYGROUND

Try a prompt against a deployment

EVALUATION

**Curated · batch · continuous
· red team**

FINE-TUNING

Train, deploy, version, compare

TRACING

Per-call latency, tokens, errors

Three surfaces, one project

Surface	Best for	Who uses it most
Foundry Portal (low-code)	Provision, browse catalog, review, govern	Decision makers · security · ops
Foundry SDK (<code>azure-ai-projects</code>)	Build the agent, run evals in code	Developers
Foundry Skills (AI-assisted, MCP)	Discover · deploy · scaffold by prompt	Anyone, fastest path

We'll use all three. Same project. Same models. Same evals.

THE FOUNDRY LIFECYCLE

From model choice to a living AI system

01

Select

*Match model to
workload*

02

Evaluate

*Test on your own
data*

03

Optimize

*Quality + latency
+ cost*

04

Operate

*Monitor · govern ·
roll back*

05

Improve

*Adopt new
models safely*

Continuous loop — every cycle improves quality, cost, and trust.

Our dev journey today

START

□ **Portal**

*create project · catalog · playground
baseline*

BUILD

□ **VS Code + SDK**

agent · router · eval · fine-tune

CLOSE

□ **Portal**

evals · red team · agent versions

Portal → Code → Portal. The loop that's natural for both speakers in this room.

PART 6

Right model, right job.

Decompose the workload. Then go shopping.

Step 1 — decompose, then shop in the catalog

Foundry catalog filter: [Sold directly by Azure](#) · [East US 2](#) · [Deployable](#)

#	Job	Pick	Why
Route	intent classify	<code>gpt-5.4-nano</code>	Sub-300ms · plenty smart for a 3-way classify
Vision	extract receipt	<code>gpt-5.4-mini</code>	Vision support · frontier overkill
Policy	"can I expense X?"	<code>gpt-5.4-mini</code> → fine-tune	Cheap base + fine-tunable
Plan	multi-step + tools	<code>gpt-5.4</code>	Frontier earns its keep here
Translate	EN ↔ DE	<code>gpt-5.4-mini</code> (reuse)	Same deployment as vision

4 of 5 jobs don't need a frontier model. *That insight is the whole talk.*

Demo — the Foundry Skill in action

```
> List Azure Direct models in East US 2 matching gpt-5.4*.
> For each: exact version · max tokens · supports fine-tuning ·
> input/output price per 1K.

> Now deploy:
>   router-nano      (gpt-5.4-nano,   TPM 10K)
>   mini-vision      (gpt-5.4-mini,   TPM 20K)
>   policy-mini-base (gpt-5.4-mini,   TPM 20K)
```

Under the hood: `models_list` + `models/deploy-model` MCP tools.

Same calls a CI pipeline would make. Same calls the portal makes.

A naming convention that pays off later

```
planner-gpt54      ← gpt-5.4  
router-nano        ← gpt-5.4-nano  
mini-vision        ← gpt-5.4-mini  
policy-mini-base   ← gpt-5.4-mini  
policy-mini-ft     ← gpt-5.4-mini (fine-tuned, coming in Part 9)
```

Deployments describe the [job](#), not the model.

When we swap in the fine-tune later, the app change is [one boolean](#).

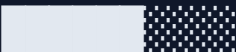
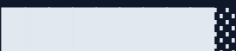
Route to the right model per task

```
intent = route(user_msg)           # router-nano    (~180ms)
if has_image:
    receipt = vision_extract(img)    # mini-vision  (~1.2s)
policy_note = policy_lookup(user_msg) # policy-mini-base
itinerary = planner.run(user_msg, ...) # planner-gpt54 + tools
```

Same app shape. Different model per call.

The router is the only piece of "agent" framework you actually need at this scale.

Scorecard — two bars move

Quality		0.61	← measured for the first time
Cost		\$0.063	↓ from \$0.108
Latency		9.4s	↓ from 12.3s

Cost down 42%. Latency down 24%.

Quality measured — and it's not where we need it yet.

PART 7

Measure what you move.

Quality unmeasured $\neq$ quality is good.

Before we move quality, we have to see it

20 curated rows **hand-written, ground truth**
the smoke test on every commit

Two evaluators
deterministic schema/constraint check + LLM-
as-judge (semantic correctness)

~200 synthetic rows **grown from the seed**
across 6 axes: geography · budget · language ·
policy traps · receipt categories · intent balance

Dataset is versioned.
No version = no comparison = no story.

Demo — synthetic data generation

```
# 20 seed rows → ~200 across axes:  
# intent · geography · budget edge · language · receipt category · policy traps  
generate("eval-seed.jsonl",  
         out_path="eval-full.jsonl",  
         n_per_axis=25)  
  
# → Wrote 198 rows (178 synthetic).
```

Then read the first 30 synthetic rows by hand.
A 150-row clean eval beats a 500-row noisy one.

Three evals, three jobs

Flavor	When you run it	What it answers
Curated (20 rows)	Every commit	"Did I break a known case?"
Batch (~200 rows)	Before each deploy	"How does this version score overall?"
Online / continuous	Production traffic, sampled	"Is real-world quality drifting?"

All three live in the same Evaluation tab. Same dataset versions. Same evaluators.

PART 8

Customize.

Fine-tune the cheap model. Beat the frontier.

When to fine-tune (and when not to)

✅ Use fine-tune when...	❌ Skip fine-tune when...
A cheap base is close but not great	You haven't tried a better prompt + retrieval
Narrow, stable domain task	The task changes weekly
≥30 high-quality labeled examples	Your eval doesn't isolate the task
The frontier is too expensive at scale	The base model already hits target

Contoso policy QA: ✅ on all four.

Demo — fine-tune `gpt-5.4-mini` for policy

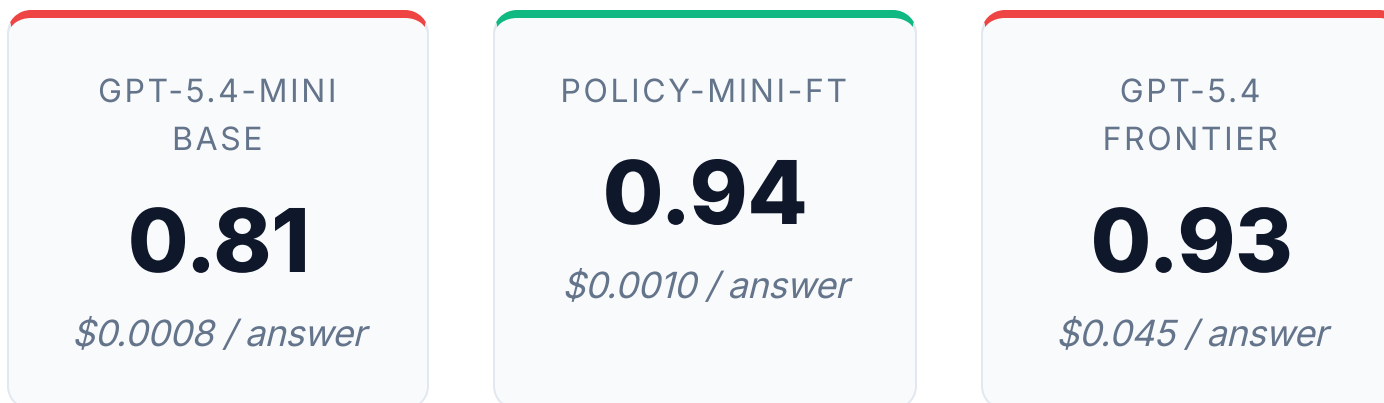
```
job = client.fine_tuning.jobs.create(  
    training_file=train.id,  
    validation_file=val.id,  
    model="gpt-5.4-mini",  
    hyperparameters={"n_epochs": 3},  
    suffix="contoso-policy-v1",  
)
```

24 train rows · 6 val rows · ~2 hours.

Then in the agent, **one line**:

```
USE_FT_POLICY = True
```

POLICY QA — BEFORE VS. AFTER



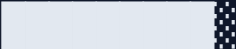
~50× cheaper than the frontier, with higher quality.

PART 9

All green.

The end-of-talk slide.

Final scorecard

Quality		0.94	✓
Cost		2.8¢	✓
Latency		7.6s	✓

All three targets met. No heroics. Just the right model for each job, measured.

The journey, on one slide

Version	Architecture	Quality	\$/task	p50 latency
v1	gpt-5.4 does everything	0.61	\$0.108	12.3s
v2	Router + per-task models	0.78	\$0.063	9.4s
v3	+ fine-tuned policy	0.94 ✓	\$0.028 ✓	7.6s ✓

*Each row = **one decision**, justified by the scorecard **before** it was made.*

THE DELTAS, END TO END

QUALITY

+54%

0.61 → 0.94

COST

-74%

*\$0.108 →
\$0.028*

LATENCY

-38%

12.3s → 7.6s

PART 10

Govern.

Back to the portal — what your CFO,
SecOps, and auditors see.

Close the loop — back in the portal

- **Evaluation tab** — v1 vs v3 side-by-side · row-level diffs · dataset version pinned
- **Red team** — jailbreak · prompt injection · sensitive content (out of the box)
- **Agent versions** — v1.0 · v2.0 · v3.0, each with bound deployments and a pinned eval
- **Tracing** — the router → vision → policy(ft) → planner chain, per-call timing
- **Continuous evaluation** — sample 5% of prod traffic, catch regressions in days, not quarters

Everything you built in code is visible, governable, and comparable here.

What we showcased today

- ✓ **Model selection** — per task, with a defensible reason
- ✓ **Synthetic dataset generation** — to grow a meaningful eval
- ✓ **Evaluations** — curated · batch · LLM-judge · red team · continuous
- ✓ **Customization** — fine-tune the cheap model, beat the frontier on cost **and** quality
- ✓ **Agent versioning** — v1 / v2 / v3 comparable side-by-side in the portal

All on Microsoft Foundry. East US 2. 7-day build. Toy-scale data.

PART 11

The takeaway, again.

*"The unit of progress is the model decision —
made per task, against a per-task scorecard."*

QUALITY

0.61 → 0.94

COST

\$0.108 →
\$0.028

LATENCY

12.3s → 7.6s

BUILD IT YOURSELF

Now you.

Repo: `nitya/model-releases-FORK` → `workshops/foundry-models-e2e/`

Plan: `.plans/foundry-models-e2e-plan.md` (full speaker script + trainer guide)

Hands-on: `workshops/foundry-models-e2e/README.md` (Steps 0–8, ~7 days of evenings)

Code: `workshops/foundry-models-e2e/code/` (`s02_...` through `s06_...`)

Region: East US 2 · **Models:** `gpt-5.4` family (Azure Direct)

Questions?